

Werkzaamheden 9/9/2025

Vandaag heb ik gewerkt aan het maken van een hamburger menu systeem voor de DIPPER-applicatie, het probleem was eerst dat de website niet mobile vriendelijk genoeg was, dit komt omdat het navigatiesysteem in de header te groot was om op mobile te weergeven, waardoor het ervoor zorgde dat de pagina veel groter was dan de telefoon zelf, dit heb ik aangepast door een mediacode erin te doen die 2 headers maakt. Degene die voor de computer gemaakt is staat normaal erop. Maar als de code detecteert dat je beeldscherm minder groot is dan 728 pixels (het hoeveelheid plek dat de nav inneemt). Dan pakt hij de mobile nav aan, de mobile nav is een hamburger menu, dit betekent dat je op een knopje rechtsboven kan klikken waardoor de normale nav systeem uitklapt, hier is de code ervoor.

```
base.html | Git Local Changes (Working Tree) - 2 of 2 changes
36
37     <div class="hamburger-menu">
38 <div class="hamburger-icon" id="hamburger-icon">
39     &#9776; <!-- Hamburger icon -->
40 </div>
41 <div class="menu-links" id="menu-links">
42     {% if current_user.is_anonymous %}
43         <a href="{{ url_for('auth.login_route') }}">Login</a>
44     {% else %}
45         <a href="{{ url_for('projects.projects') }}">Projects</a>
46         <a href="{{ url_for('research_projects.research_projects') }}">Research projects</a>
47         <a href="{{ url_for('researchers.researchers') }}">Researchers</a>
48         <a href="{{ url_for('courses.course') }}">Courses</a>
49         {% if current_user.admin %}
50             <a href="{{ url_for('configuration.configuration') }}">Configuration</a>
51         {% endif %}
52         <a href="{{ url_for('downloads.downloads') }}">Download data</a>
53         <a href="{{ url_for('auth.profile_route') }}">Profile</a>
54         <a href="{{ url_for('auth.logout_route') }}">Logout</a>
55     {% endif %}
56 </div>
57
58 <script>
59 const hamburgerIcon = document.getElementById('hamburger-icon');
60 const menuLinks = document.getElementById('menu-links');
61
62 hamburgerIcon.addEventListener('click', () => {
63     menuLinks.style.display = menuLinks.style.display === 'flex' ? 'none' : 'flex';
64 });
65 </script>
66
67
69 </nav>
```

```

/ Responsive Table /
@media (max-width: 768px) {
  .nav-links{
    display: none;
  }

  .hamburger-menu {
    position: relative;
    display: grid
  }

  .hamburger-icon {
    font-size: 30px;
    cursor: pointer;
    user-select: none;
  }

  .menu-links {
    display: none;
    flex-direction: column;
    position: absolute;
    top: 40px; /* Adjust based on your layout */
    right: 40%;
    background-color: #fff; /* Menu background */
    border: 1px solid #ccc;
    padding: 10px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
    z-index: 1000;
    width: 250px;
    font-size: 22px;
  }

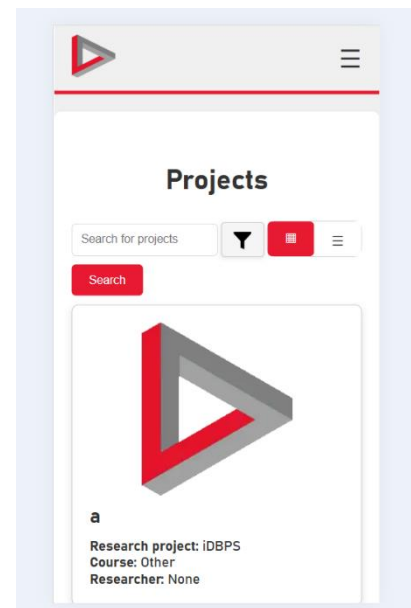
  .menu-links a {
    padding: 8px 12px;
    text-decoration: none;
    color: #000;
  }

  .menu-links a:hover {
    background-color: #f0f0f0;
  }
}

```

Werkzaamheden 09-19 september '25

Deze week was ik verdergegaan met het maken van de mobile interface, het was van mij gevraagd om het wat meer mobile vriendelijk te maken en functioneel, dus deed ik dat door de buttons te veranderen naar een iets gebruikersvriendelijker design. Dit is dus hoe het nu uit ziet. Zoals je kan zien heb ik de buttons aangepast. Hier is de code ervoor. Ik moet volgende week als ik alles af heb een git change doen en dat documenteren, deze week ging het een beetje sloom. Merendeels omdat ik zat uit te vogelen waar wat stond, omdat de code best verwarrend kan zijn.



```

</div> -->
<div class="projectsearchbar">
  <form class="Form-Project" method="GET" action="{{ url_for('projects.projects') }}">
    <div class="input-group mb-3">
      <input type="text" class="form-control" name="query" placeholder="Search" />
      <div class="filter-container">
        <button type="button" class="filter-btn" onclick="toggleFilter()">
          
        </button>
        <div id="filter-options" class="filter-options">
          <button type="button" onclick="selectFilter('any', this)">Any field</button>
          <button type="button" onclick="selectFilter('title', this)">Name</button>
          <button type="button" onclick="selectFilter('projectcode', this)">Project</button>
          <button type="button" onclick="selectFilter('research_project', this)">Research</button>
          <button type="button" onclick="selectFilter('course', this)">Course</button>
          <button type="button" onclick="selectFilter('researcher', this)">Researcher</button>
          <button type="button" onclick="selectFilter('students', this)">Students</button>
        </div>
      </div>
      <!-- Hidden input so you can still submit the chosen value in a form -->
      <input type="hidden" name="field" id="selectedField" value="any">
    </form>
  </div>
</div>

<!-- JavaScript -->
<script>
function toggleFilter() {
  const menu = document.getElementById("filter-options");
  menu.style.display = menu.style.display === "block" ? "none" : "block";
}

function selectFilter(value, btn) {
  // update hidden input
  document.getElementById("selectedField").value = value;

  // update button text
  document.querySelector(".filter-btn").innerText =
    "Filter: " + btn.innerText + " ▼";

  // clear previous selection
  document.querySelectorAll(".filter-options button").forEach(b => {
    b.classList.remove("selected");
  });

  // highlights the selected option
  btn.classList.add("selected");
}

// close dropdown when you click on it
document.getElementById("filter-options").style.display = "none";

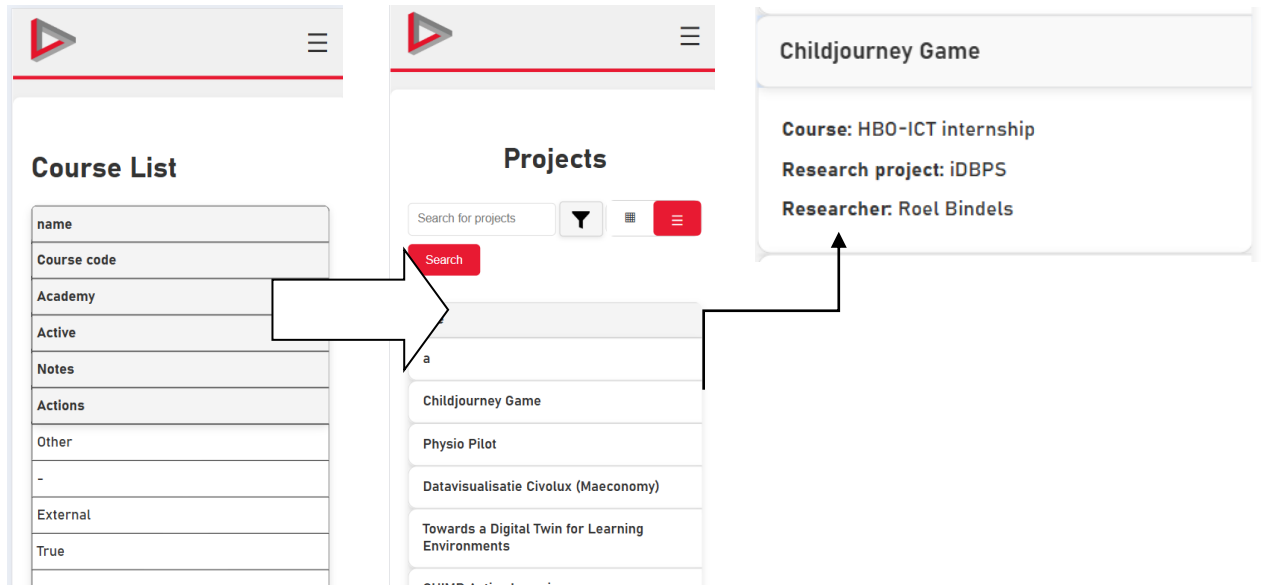
// close dropdown when you click outside
document.addEventListener("click", function(event) {
  if (!event.target.closest(".filter-container")) {
    document.getElementById("filter-options").style.display = "none";
  }
});
</script>

<button class="btn btn-primary" type="submit">Search</button>
</div>

```

Overig

Overig heb ik nog gewerkt aan de mobile design van de tabellen zelf, de grid is meerendeels hetzelfde gebleven maar de tabellen zagen er heel chaotisch uit wat ik moest fixen. Ik moet dit voor volgende week nog toevoegen aan alle paginas omdat ze allemaal anders zijn maar dit is ongeveer hoe het eerst uitzag vs hoe het eruit gaat zien

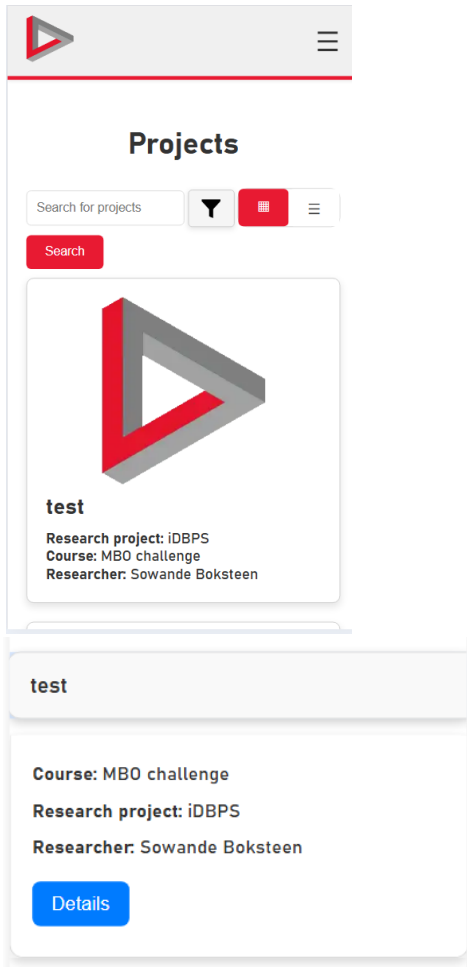


Wat ik nog moet doen

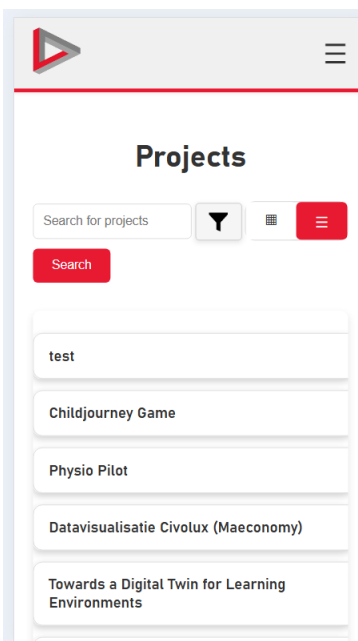
Voor de volgende week moet ik alle paginas updaten zodat het mobile design goed past overal en hetzelfde er uit ziet, zodat het makkelijker is om te overzien en fijner is om te gebruiken.

22-30 September

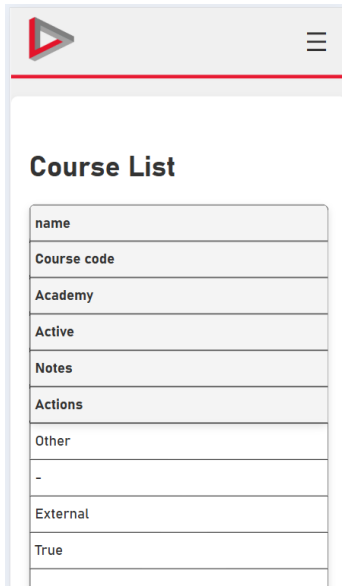
Deze week heb ik gewerkt aan het afmaken van het mobile design van DIPPER, ik heb succesvol alle paginas aangepast zodat het anders er uit ziet op mobile. Ook heb ik al een begin gemaakt aan het maken van de required knop, deze is nu nog niet functioneel. Maar hier ga ik de volgende week aan werken. Ik ben nu merendeels klaar met het mobile design als er geen andere vragen voor zijn. Dus dit laat ik hier zien



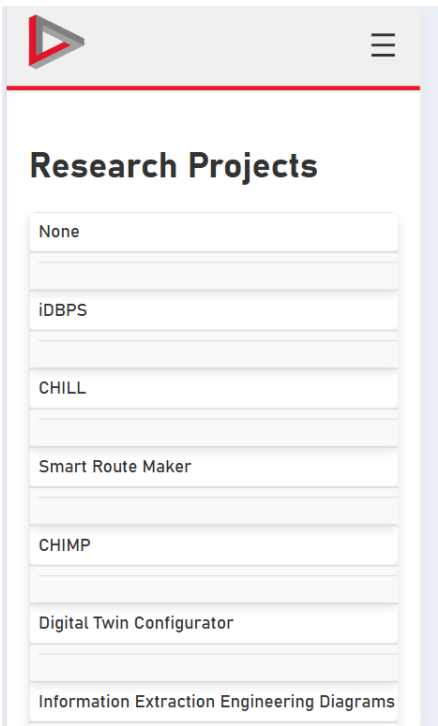
Figuur 4 wat er gebeurt als je op de home pagina test klikt



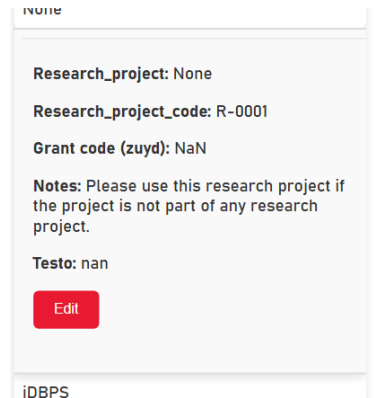
Figuur 3 Home pagina



Figuur 5 hoe hij er voor er uit zag



Figuur 2 Mobile design voor alle andere paginas (geen verschil tussen



Figuur 1 wat er gebeurt als je op de mobile title klikt op elke andere pagina

06/10/2025

Deze week heb ik gewerkt aan het maken van de required system, eerst probeerde ik een knop toe te voegen wat de configurations kon verwijderen, maar blijkbaar kon dat volgens finn alberts niet. Dus moest ik daar mee stoppen. Wat ik dus wel heb gedaan is het maken van een methode waarop gebruikers kunnen toevoegen dat een veld als required moest staan, dit klinkt makkelijk maar is iets pittiger door het feit dat deze velden dynamisch aangemaakt worden, dus het is niet zo simpel als gewoon required achter een veld tekst in html zetten. Hier is de code er voor

```
def validate_required_fields(collection_name, form_data, files=None):

    # Checks all configs for a collection and ensures required fields are not empty or not filled in.
    # Returns true or none if valid (if someone filled it in), False or error_message if invalid (if someone didn't fill it in).

    configs = db.configurations.find({
        "ConnectedCollection": collection_name,
        "inuse": True,
        "required": True
    })

    for config in configs:
        field_name = config["name"]

        # text
        if config["type"] in ["String", "ObjectId", "Date"]:
            value = form_data.get(field_name)
            if not value or value.strip() == "":
                return False, f"The field '{field_name}' is required and cannot be empty."

        # numbers
        elif config["type"] in ["Integer", "Double"]:
            value = form_data.get(field_name)
            if not value or value.strip() == "":
                return False, f"The field '{field_name}' is required and must be a number."

99
100
101     # single boolean checkbox
102     elif config["type"] == "Boolean":
103         value = form_data.get(field_name)
104         if not value:
105             return False, f"The checkbox '{field_name}' must be checked."
106
107     # array dropdown
108     elif config["type"] == "Array":
109         values = form_data.getlist(field_name)
110         if not values or len(values) < 1:
111             return False, f"You must select at least one option for '{field_name}'."
112
113     # Multiple checkboxes
114     elif config["type"] == "Checkboxes":
115         values = form_data.getlist(field_name)
116         if not values or len(values) < 1:
117             return False, f"You must choose at least one option for '{field_name}'."
118
119     # File upload (pictures etc)
120     elif config["type"] == "Binary Data":
121         if not files or field_name not in files or files[field_name].filename == "":
122             return False, f"You must upload a file for '{field_name}'."
123
124     return True, None
125
```

```

227
228 @bp.route("/add_course", methods=["POST"])
229 @admin_required
230 def add_course():
231     is_valid, error = validate_required_fields("course", request.form, request.files)
232     if not is_valid:
233         return render_template("error.html", error_message=error)
234
235     db.course.insert_one({
236         "name": request.form["name"],
237         # add other form fields if you have them
238     })
239     return redirect(url_for("course.list"))
240
241 @bp.route("/add_project", methods=["POST"])
242 @admin_required
243 def add_project():
244     is_valid, error = validate_required_fields("projects", request.form, request.files)
245     if not is_valid:
246         return render_template("error.html", error_message=error)
247
248     db.projects.insert_one({
249         "title": request.form["title"],
250         "projectcode": request.form["projectcode"],
251         # also include any dynamic config values you want to save
252     })
253     return redirect(url_for("projects.list"))
254
255 @bp.route("/add_research_project", methods=["POST"])
256 @admin_required
257 def add_research_project():
258
259     @bp.route("/add_research_project", methods=["POST"])
260     @admin_required
261     def add_research_project():
262         is_valid, error = validate_required_fields("course", request.form, request.files)
263         if not is_valid:
264             return render_template("error.html", error_message=error)
265
266         db.research_projects.insert_one({
267             "research_project": request.form["research_project"],
268             "research_project_code": request.form["research_project_code"],
269
270         })
271         return redirect(url_for("research_projects.list"))
272
273     @bp.route("/add_researcher", methods=["POST"])
274     @admin_required
275     def add_researcher():
276         is_valid, error = validate_required_fields("course", request.form, request.files)
277         if not is_valid:
278             return render_template("error.html", error_message=error)
279
280         db.researchers.insert_one({
281             "name": request.form["name"],
282             # other researcher fields if needed
283         })
284         return redirect(url_for("researchers.list"))

```

```

<script> // javascript for the client side validation of required configuration fields ^^^
document.addEventListener('DOMContentLoaded', function () {
    // attaches to any form that contains config groups
    document.querySelectorAll('form').forEach(function(form) {
        if (!form.querySelector('.config-group')) return; // skips forms without our groups

        form.addEventListener('submit', function (e) {
            // remove previous inline error messages
            form.querySelectorAll('.config-validation-error').forEach(function(n){ n.remove(); });

            const groups = form.querySelectorAll('.config-group[data-required="1"]');
            for (let gi = 0; gi < groups.length; gi++) {
                const group = groups[gi];
                const name = group.dataset.configName;
                const type = group.dataset.configType;

                let valid = true;
                let message = '';

                if (type === 'Checkboxes') {
                    const inputs = group.querySelectorAll(`input[name="${name}[]"]`);
                    const oneChecked = Array.from(inputs).some(i => i.checked);
                    if (!oneChecked) {
                        valid = false;
                        message = `Please select at least one option for "${name}".`;
                    }
                } else if (type === 'Array') {
                    const sel = group.querySelector(`select[name="${name}"]`);
                }

                } else if (type === 'Binary Data') {
                    const fileInput = group.querySelector(`input[type="file"][name="${name}"]`);
                    if (!fileInput || !fileInput.files || fileInput.files.length === 0) {
                        valid = false;
                        message = `Please upload a file for "${name}".`;
                    }
                } else if (type === 'Boolean') {
                    const cb = group.querySelector(`input[type="checkbox"][name="${name}"]`);
                    if (!cb || !cb.checked) {
                        valid = false;
                        message = `The checkbox "${name}" must be checked.`;
                    }
                } else {
                    // for the strings integers doubles dates and objectids
                    const input = group.querySelector(`input[name="${name}"]`);
                    const val = input ? input.value : '';
                    if (!val || String(val).trim() === '') {
                        valid = false;
                        message = `The field "${name}" is required and cannot be empty.`;
                    }
                }
            }

            if (!valid) {

```



```

111     }
112
113     if (!valid) {
114         e.preventDefault();
115         // inserts the error message just above the input field
116         const err = document.createElement('div');
117         err.className = 'config-validation-error';
118         err.style.color = 'red';
119         err.style.marginBottom = '8px';
120         err.textContent = message;
121         group.parentNode.insertBefore(err, group);
122         group.scrollIntoView({ behavior: 'smooth', block: 'center' });
123         return; // stops it at first validation error
124     }
125     } // end for groups
126 }); // submit listener
127 }); // forms loop
128 });
129 </script>
130 {% endmacro %}
131
132
133
134

```

Dit is de functionaliteit van de checkmark, de front end is ook aan het einde te zien in de javascript. Het ziet er dus zo uit.

The field "Test1234" is required and cannot be empty.

Test1234 *:

Please select a value for "HalloweenId123".

HalloweenId123 *:

Please select at least one option for "3fsd".

3fsd *:

- ☐ s
- ☐ d
- ☐ m
- ☐ r

The field "Notes" is required and cannot be empty.

Notes *:

Notes *: